

4OPT1: Numerical Project in Python

Portfolio selection

Andrea Simonetto

version: September 22, 2025

We will consider the problem of finding the best portfolio selection among a list of assets. The project will make you experience several numerical methods and software to solve optimisation problems.

This project counts 40% of your grading. The project will be graded as follows: 10 points for the technical exactness; 10 points for the clarity, formatting, and readability of the report; 10 points for the graphs (font size, explanations, selections).

Some instructions.

- The project can be done in pairs.
- After the first three hours, you need to submit what you have done at that point via the Moodle page of the course. Name the file: `LastName1_LastName2_Project_After_three_hours.py`
- By **December, Wednesday 3, at 13H00 Paris time**, you need to submit a report in a single .pdf file, of no more than 8 pages, that is readable, and it has inside all the elements to assess your work. Name the file: `LastName1_LastName2_Project_Final.pdf`

You also need to submit a python script that we can run to assess the correctness of your plots and graphs. The script will have to generate the results that you have in the report. And it has to be readable. Name the file: `LastName1_LastName2_Project_Final.py`

No extension allowed, No resit allowed.

- As you figured already, you will use python, and in particular you will need the following packages (please refrain from using any other packages that are not strictly needed).

```
import numpy as np
import matplotlib.pyplot as plt
import cvxpy as cp
import scipy.optimize as sp
import pandas as pd
import yfinance as yf
```

- Do as much as you can during the TP session (3 hour) at ENSTA, but it is possible that you will need to do extra work in addition to it.
- Don't wait the last day to put the project together.

Writing a readable report is very important in life. The readability of the report will be very important for the final grade, as said. Some typesetting instructions follow.

1. Save and incorporate **figures** and graphs in **.pdf** format. (No .jpeg, no .png, no print-screen). In order to help you generate readable graphs, you have access to a setup example python script. It saves a sample figure in .pdf, and so must you.
2. The labels and legends of the graph should have a fontsize similar to the one used in the main text.
3. Equations should be punctuated. They are objects living in your text, so add commas, and full-stops.
4. Don't start a sentence with a symbol.
5. Don't mix French with English.
6. Don't put code in the text, or print-screen of results. Especially with black background.

1 Getting started: portfolio selection

We consider a classical portfolio problem with n assets or stocks held over a period of time. We let x_i denote the amount of asset i held throughout the period, with x_i in euro, at the price at the beginning of the period. A normal long position in asset i corresponds to $x_i > 0$; a short position in asset i (i.e., the obligation to buy the asset at the end of the period) corresponds to $x_i < 0$. We let p_i denote the relative price change of asset i over the period, i.e., its change in price over the period divided by its price at the beginning of the period. The overall return on the portfolio is $r = p^\top x$ (given in euro, and where we have stacked the x_i and p_i in vectors). The optimisation variable is the portfolio vector $x \in \mathbb{R}^n$.

A wide variety of constraints on the portfolio can be considered. The simplest set of constraints is that $x_i \geq 0$ (i.e., no short positions) and $\mathbf{1}^\top x = B$ (i.e., the total budget to be invested is B , which is often taken to be one). We take a stochastic model for price changes: $p \in \mathbb{R}^n$ is a random vector, with known mean μ and covariance Σ . Therefore the return r is a (scalar) random variable with mean $\mu^\top x$ and variance $x^\top \Sigma x$. The choice of portfolio x involves a trade-off between the mean of the return, and its variance.

1.1 Markowitz portfolio problem

The classical portfolio optimisation problem, introduced by Markowitz, is the QP

$$\begin{aligned} & \underset{x \in \mathbb{R}^n}{\text{minimise}} && x^\top \Sigma x \\ & \text{subject to} && \mu^\top x \geq r_{\min}, \\ & && \mathbf{1}^\top x = 1, \\ & && x \geq 0, \end{aligned} \tag{1}$$

with $r_{\min}$ an expected minimal return. The problem is convex.

2 CVXPY

CVXPY is an open source Python-embedded modeling language for convex optimisation problems. It lets you express your problem in a natural way that follows the math, rather than in the restrictive standard form required by solvers.

In the following, you will explore the language and use it to solve some optimisation problems.

2.1 A simple least-squares problem

Question 1 (1pt) We start simple. Have a look at the documentation in python and solve the following optimisation problem,

$$\begin{aligned} & \underset{x \in \mathbb{R}^n}{\text{minimise}} && \frac{1}{2} \|Ax - b\|_2^2 \\ & \text{subject to} && 0 \leq x \leq 1, \end{aligned} \tag{2}$$

where $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$ are randomly generated. Take,

```
n = 20
m = 30
np.random.seed(1)
A = np.random.randn(m, n)
b = np.random.randn(m)
```

to start.

Question 2 (2pt) Re-solve the above problem by using CVXPY with different n, m . Plot the computational time to solve the problem as a function of n and m as they grow from $n = 20$ and $m = 30$ to $n = 2000$ and $m = 3000$.

Question 3 (2pt) Re-solve the above problem by using `SCIPY.optimize` (another solver) with the solver: `SLSQP` and different n, m . Plot the computational time to solve the problem as a function of n and m as they grow from $n = 20$ and $m = 30$ to $n = 2000$ and $m = 3000$. Is the solution you obtain the same as CVXPY? Is the computational time similar?

2.2 Solve the Markowitz portfolio problem

We now use CVXPY to solve the Markowitz portfolio problem (Problem (1)).

Use the code provided to load the training data and test data with a few assets (for now 3):

```
Ytrain, Ytest = download_finance_data(n_assets=3)
mu, sigma = compute_moments(Ytrain)
x = markovitz_portfolio(mu, sigma)
```

Note we train the best weights on 4 years (2016-2019) and then we will test on (2020).

Question 4 (2 points) Your job is to code the python function `markovitz_portfolio(mu, sigma)` that solves Problem (1) and return the optimal x . Use $r_{\min} = 1.6/252$, so that your portfolio should yield more than a Livret A.

Question 5 (2 points) Use the function,

```
s_tr, s_te = compute_metrics(x, Ytrain, Ytest)
```

to understand the expected return and its variance on the training and test data. What do you observe? Increase the assets up to 20. Note that the results are given in %.

2.3 Probabilistic variant

We assume now that the price change vector $p \in \mathbb{R}^n$ is a **Gaussian** random variable, with mean μ and covariance Σ . Therefore the return r is a Gaussian random variable with mean $r = \mu^\top x$ and variance $\sigma_r^2 = x^\top \Sigma x$. Consider a loss risk constraint of the form

$$\mathbf{prob}(r \leq \alpha) \leq \beta,$$

where α is a given unwanted return level (e.g., a large loss) and β is a given maximum probability. We can express this constraint using the cumulative distribution function Φ of a unit Gaussian random variable. The inequality above is equivalent to

$$\mu^\top x + \Phi^{-1}(\beta) \|\Sigma^{1/2} x\|^2 \geq \alpha.$$

The problem is now,

$$\begin{aligned} & \underset{x \in \mathbb{R}^n}{\text{minimise}} && x^\top \Sigma x \\ & \text{subject to} && \mu^\top x + \Phi^{-1}(\beta) \|\Sigma^{1/2} x\|^2 \geq \alpha \\ & && x \geq 0, \mathbf{1}^\top x = 1. \end{aligned} \tag{3}$$

Question 6 (2 points) Use $\beta = 0.49$ and $\alpha = 1.6/252$. The $\Phi^{-1}(\beta)$ is the inverse cumulative distribution function, given in python as `scipy.stats.norm.ppf(0.49)`.

Code a new solver,

```
x = markovitz_portfolio_probabilistic(mu, sigma)
```

that solves Problem (3). Analyse the results with the training and testing data. Compare with the standard version (Problem (1)).

Increase the number of assets and reduce β . What do you observe?

Question 7 (1 point) The testing set is limited to 2020, which was an odd year due to COVID. How the results change taking the 2020 and 2021?

2.4 Plotting

Let us now analyse the results with some plotting.

Question 8 (1 point) Plot the time-series data of the assets (use 3 assets for clarity) in the training and testing phase. The time-series are stored in `Ytrain`, `Ytest`. Be careful to readability.

To further get a deeper look in the results, we perform a Montecarlo analysis: we run plenty of samples of how the assets could have behaved and plot the result in a standard deviation vs. return plane. This is done by the functions,

```
n_samples = 1000
```

```
n_assets = 3
```

```
c_mean, c_var = montecarlo_sim(n_assets, n_samples, Ytrain)
scatter_plot_port(c_mean, c_var, s_tr.iloc[0], s_tr.iloc[1])
```

```
c_mean, c_var = montecarlo_sim(n_assets, n_samples, Ytest)
scatter_plot_port(c_mean, c_var, s_te.iloc[0], s_te.iloc[1])
```

Question 9 (2 points) Use the above functions and plot the results you have obtained previously. Comment on what you observe. Change the number of assets to 20 and do the same.

You may want to use also the function `portfolio_composition_plot(Ytrain, x)` for a pie chart of the portfolio selection. In this case, you need to install the package `riskfolio-lib`.

3 Kurtosis

In some cases, minimising the variance is not enough to obtain a good portfolio. Sometimes, one wants to minimise higher moments. For example, the fourth-order moment, i.e., the kurtosis, is often chosen in this case.

The problem then reads,

$$\begin{aligned}
& \underset{x \in \mathbb{R}^n}{\text{minimise}} && x^\top M_4 (x \otimes x \otimes x) \\
& \text{subject to} && \mu^\top x \geq r_{\min}, \\
& && \mathbf{1}^\top x = 1, \\
& && x \geq 0,
\end{aligned} \tag{4}$$

where $M_4 \in \mathbb{R}^{n \times n \times n \times n}$ is the kurtosis tensor and $\otimes$ is the Kroenecker product. This is a complicated nonconvex problem.

A slightly different formulation is offered as,

$$\begin{aligned}
& \underset{\substack{x \in \mathbb{R}^n, X \in \mathbb{S}(\mathbb{R}^n), \\ z \in \mathbb{R}^{n(n+1)/2}, g \in \mathbb{R}}}{\text{minimise}} && g \\
& \text{subject to} && X = xx^\top \\
& && z = L_2 \text{vec}(X) \\
& && \left\| [S_2 \Sigma_4 S_2^\top]^{1/2} z \right\|_2 \leq g \\
& && \sum_{i=1}^n x_i = 1 \\
& && x \geq 0 \\
& && \mu^\top x \geq r_{\min},
\end{aligned} \tag{5}$$

where the coefficients L_2, S_2, Σ_4 are problem specific (and given to you).

This is also a nonconvex problem.

Question 10 (2pt) *What makes Problem (5) nonconvex?*

4 Sequential quadratic programming

4.1 Theory

To solve nonconvex problems, we turn to one of the most used methods in this respect: sequential quadratic programming (SQP), or the upgraded SLSQP (sequential least-squares quadratic programming). We haven't seen it in class, let's look at it now.

SQP is an iterative method for solving constrained differentiable optimisation problems. The basic idea behind this algorithm is that one considers a sequence of quadratic problems (which can be solved with some existing solver) whose KKT-pairs converge toward a KKT-pair of the initial optimisation problem (indicating thus that one might have there a local optimal solution to that problem).

Consider the nonconvex problem,

$$\begin{aligned}
& \underset{x \in \mathbb{R}^n}{\text{minimise}} && f(x) \\
& \text{subject to} && h(x) = 0, g(x) \leq 0,
\end{aligned} \tag{6}$$

with $f : \mathbb{R}^n \rightarrow \mathbb{R}$, $h : \mathbb{R}^n \rightarrow \mathbb{R}^m$, $g : \mathbb{R}^n \rightarrow \mathbb{R}^p$. We assume that all the functions are at least differentiable and that the cost is doubly differentiable. We don't assume convexity.

Take now an initial starting point x^0 . This point doesn't have to be feasible. Take now a Taylor expansion of the problem around a point x^k as $x^k + \delta x$:

$$\begin{aligned}
& \underset{\delta x \in \mathbb{R}^n}{\text{minimise}} && f(x^k) + \langle \nabla f(x^k), \delta x \rangle + \frac{1}{2} \langle \nabla^2 f(x^k) \delta x, \delta x \rangle \\
& \text{subject to} && h(x^k) + \langle \nabla h(x^k), \delta x \rangle = 0, g(x^k) + \langle \nabla g(x^k), \delta x \rangle \leq 0.
\end{aligned} \tag{7}$$

This last problem has affine constraints in δx , and a quadratic cost. If $\nabla^2 f(x^k) \succeq 0$, then the problem is convex, and a quadratic program¹.

So one could think of having a sequence of problems:

1. Set x^0 . For all $k = 0, 1, \dots, k_{\max}$ do:
 - (a) Solve problem (7) for δx ,
 - (b) Set $x^{k+1} = x^k + \delta x$

This is the core of the SQP method.

Under some mild conditions the sequence $\{x^k\}$ converges to a local minimiser of the original nonconvex problem. (Hurray!)

Variants. Variants of the basic algorithm (invented in the 70's) exist. First, one could add a trust region for δx to limit large steps. For instance, you could add $\|\delta x\| \leq \Delta^k$, with the latter Δ^k to tune.

If one has convex but nonlinear constraints, one could also leave them as they are and not linearise them. If the constraint or cost are convex, but not specifically affine (resp. quadratic), we call the method sequential convex programming (SCP).

4.2 A simple nonconvex problem

Consider a simple nonconvex problem of the form,

$$\begin{aligned} & \underset{x \in \mathbb{R}^n}{\text{minimise}} && \frac{1}{2} \|Ax - b\|_2^2 \\ & \text{subject to} && 0 \leq x \leq 1, \|x\|_2^2 \geq 1/2 \end{aligned} \tag{8}$$

where $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$ are randomly generated. Take,

```
n = 20
m = 30
np.random.seed(1)
A = np.random.randn(m, n)
b = np.random.randn(m)
```

Question 11 (3 points) Code a SQP solver for the above problem and use CVXPY for each convex subproblem. Plot the relevant quantities (for instance the norm of the residual) as they converge (or not) to a stationary point. Are the constraints satisfied?

Question 12 (1 point) Compare your code with SCIPY.optimize and the solver SLSQP in terms of speed, iterations to convergence, and quality of the final solution. What do you observe?

4.3 Portfolio kurtosis problem

Look now at the portfolio kurtosis problem (5). Use the instructions,

```
D2, L2, S4 = compute_coefficients(Ytrain, n_assets)
S2 = D2.T @ D2 @ L2
```

¹If the Hessian is indefinite we can choose to remove it, or add a regularisation. It won't matter for us here.

to generate the matrices you need as coefficients. Consider the perturbation problems,

$$\begin{aligned}
& \underset{\substack{\delta x \in \mathbb{R}^n, \delta X \in \mathbb{S}(\mathbb{R}^n), \\ \delta z \in \mathbb{R}^{n(n+1)/2}, \delta g \in \mathbb{R}}}{\text{minimise}} & & g^k + \delta g \\
& \text{subject to} & & X^k + \delta X = x^k x^{k,\top} + 2x^k \delta x^\top \\
& & & z^k + \delta z = L_2 \text{vec}(X^k + \delta X) \\
& & & \left\| [S_2 \Sigma_4 S_2^\top]^{1/2} (z^k + \delta z) \right\|_2 \leq g^k + \delta g \\
& & & \sum_{i=1}^n x_i^k + \delta x_i = 1 \\
& & & x^k + \delta x \succeq 0 \\
& & & \mu^\top (x^k + \delta x) \geq r_{\min}, \\
& & & X^k + \delta X \succeq 0,
\end{aligned} \tag{9}$$

and the update $\vec{x} = \vec{x}^k + \delta \vec{x}$, where $\vec{x}$ collects all the decision variables.

The perturbation problems are convex (we have used the SCP variant of SQP) and can be solved with CVXPY.

Question 13 (4 points) *Code a SCP method to solve the portfolio kurtosis problem (5) by leveraging the perturbation problems (9). Use CVXPY to solve the perturbation problems. Test your code with different initial conditions and plot relevant quantities to assess the quality of the solution, with respect to the minimum variance portfolio.*

Suggestions. Use

```
from scipy.linalg import sqrtm
Sigma_4_sqrt = sqrtm(S2 @ S4 @ S2.T)
```

to compute the square root of the matrix. Use `cp.SOC` for the norm constraint.

Use `zk+dz == L2 @ cp.reshape(cp.vec(Xk + dX), (n * n, 1))` for the constraint in z and X .

Also, use `xk = xk+np.array(dx.value)` and similar for the iterative process.

Question 14 (2 points) *Solve the portfolio kurtosis problem (5) with `SCIPY.optimize` and the `SLSQP` solver. Compare the results with your code in terms of speed and quality of the obtained solution.*

Question 15 (1 point) *Plot the resulting portfolios with Montecarlo and pie charts.*

5 Convex relaxation

Problem (5) can be relaxed as a SDP by changing the constraint $X = xx^\top$ into $X \succeq xx^\top$, which can be written as,

$$\begin{bmatrix} X & x \\ x' & 1 \end{bmatrix} \succeq 0,$$

via Schur's complement.

Then the convex relaxation of problem (5) is,

$$\begin{aligned}
& \underset{\substack{x \in \mathbb{R}^n, X \in \mathbb{S}(\mathbb{R}^n), \\ z \in \mathbb{R}^{n(n+1)/2}, g \in \mathbb{R}}}{\text{minimise}} && g \\
& \text{subject to} && \begin{bmatrix} X & x \\ x^\top & 1 \end{bmatrix} \succeq 0 \\
& && z = L_2 \text{vec}(X) \\
& && \left\| [S_2 \Sigma_4 S_2^\top]^{1/2} z \right\|_2 \leq g \\
& && \sum_{i=1}^n x_i = 1 \\
& && x \geq 0 \\
& && \mu^\top x \geq r_{\min},
\end{aligned} \tag{10}$$

Question 16 (2 points) *How are the two problems (5) and (10) related? When solving the latter solves the former as well? In general, is the optimal value of (10) less, equal, or greater than the one of (5)?*

Question 17 (3 points) *Solve Problem (10) with CVXPY for a number of different assets. Provide comparisons with the previous methods in terms of computational time, and quality of the results on training and testing data.*

Check on the constraint $X = xx^\top$. Is it satisfied with the convex relaxation? What can you say then?

6 Conclusions

Question 18 (1 point) *Provide some concluding thoughts.*

Bonus. *Where machine learning can be used to improve the return/variance of your portfolio and why is it very hard to do so?*